

SIMATS ENGINEERING

Department of Computer Science & Engineering
CSA12– Computer Architecture for Multicore Systems

NAME:M.Dinesh kumar

REG NO:192525298

Assessment Tool 1 – Industry Problem Solving Task (Annexure A) – Answers

Q1. Smartphone Performance Optimization

Problem Statement: A mobile manufacturer observes lag while switching between apps. Analyze the role of L1, L2, L3 cache and DRAM and propose a hierarchical memory redesign to reduce latency and improve throughput.

1. Understanding of the Industry Problem

Modern smartphone SoCs run dozens of apps in various states (foreground, background-suspended, cached). Lag during app-switching is usually not a raw CPU-speed issue but a memory-hierarchy issue: when a user switches apps, the working set of the new app is 'cold' in the caches, forcing repeated cache misses and DRAM (LPDDR) fetches before the UI can render smoothly, producing visible jank/frame drops.

2. Role of the Memory Hierarchy

- **L1 Cache:** Smallest (32–64 KB, split I/D), per-core, single-digit-cycle latency. Holds the instructions/data of the thread actually executing. On an app switch the L1 is almost entirely invalidated for the new app's context, so the first instructions always miss to L2/L3.
- **L2 Cache:** 256 KB–1 MB, usually private per-core or shared within a core cluster (e.g., ARM DynamIQ cluster), tens-of-cycles latency. Buffers the recently used working set of the whole app; determines how much of the previous app's state survives a quick switch back.
- **L3 / SLC (System Level Cache):** 2–8 MB, shared across all CPU/GPU/NPU clusters, acts as a 'victim cache' that retains recently evicted lines from L1/L2. This is the key level for multitasking because it can hold fragments of several apps at once, cutting the number of full DRAM round-trips on a switch.
- **DRAM (LPDDR5/5X):** Main memory holding the complete state of every open app (including background ones). Access latency (~100+ ns) is roughly 50–100× that of L1, so every L3 miss during an app switch is expensive and is the dominant source of perceived lag.

3. Proposed Hierarchical Memory Redesign

- **Enlarge and partition the SLC (L3):** Increase shared L3/SLC capacity and use cache QoS/way-partitioning so the foreground app gets a guaranteed slice while recently used background apps retain a smaller reserved portion, reducing eviction thrash.
- **Context-aware prefetching:** Use the OS task-switcher's recency history to prefetch the likely-next app's critical working set from DRAM into L3 just before the user completes a swipe gesture (predictive prefetch), hiding DRAM latency behind user interaction time.
- **Faster, higher-bandwidth DRAM:** Move to LPDDR5X/LPDDR6-class DRAM with higher bandwidth and lower per-access energy so that unavoidable DRAM fetches complete faster and with less power impact.

- **Compressed RAM (zRAM) with hardware decompression:** Keep more background apps resident in DRAM (compressed) rather than killing/swapping them to flash storage, so 'switch back' is a fast decompress-from-DRAM operation instead of a slow storage read.
- **Core-cluster-aware L2 sizing:** Assign foreground-critical threads to the cluster with the larger private L2 so UI-thread instructions/data stay resident longer.

4. Expected Outcome

This redesign shortens the effective memory access path during app switches (fewer full DRAM misses), increases the hit rate for the immediately-needed working set, and therefore reduces latency and improves sustained throughput/frame consistency during multitasking.

Q2. Cloud Server Memory Bottleneck

Problem Statement: A cloud service provider experiences high latency in database queries. Compare cache hierarchy vs virtual memory paging and recommend improvements using appropriate memory technologies.

1. Understanding of the Industry Problem

Database servers handle working sets (indexes, hot rows, buffer-pool pages) far larger than on-chip cache but expect near-memory-speed access. When the working set exceeds available cache and DRAM, the OS falls back on virtual-memory paging to disk, and each page fault costs milliseconds instead of nanoseconds — this is almost always the real source of 'high latency' at scale.

2. Cache Hierarchy vs Virtual Memory Paging

- **Cache hierarchy (L1/L2/L3):** Hardware-managed, automatic, extremely fast (ns-scale) but capacity-limited (a few MB to tens of MB). Effective only while the hot dataset (indexes, frequently-joined rows) fits within it; beyond that, hit rate collapses and every access pays full DRAM latency.
- **Virtual memory paging:** OS/software-managed, allows a working set far larger than physical RAM by backing pages on secondary storage. Capacity is huge, but a page fault requires a context switch plus a disk/SSD access — 4–5 orders of magnitude slower than a cache hit. Under high concurrency this causes thrashing, which is the classic cloud-database latency killer.

3. Recommendations Using Appropriate Memory Technologies

- **Provision sufficient DRAM:** Size DRAM (DDR5 ECC RDIMMs) so the entire hot working set/buffer pool fits in main memory, eliminating disk-backed paging for normal operation.
- **Use huge pages (2 MB/1 GB):** Reduce TLB misses and page-table walk overhead for large in-memory datasets, cutting virtual-to-physical translation latency.
- **NUMA-aware allocation:** On multi-socket servers, pin database processes and their memory to the local NUMA node to avoid slow cross-socket memory access.
- **Persistent/tiered memory (e.g., CXL-attached memory or NVDIMM):** Insert a tier between DRAM and SSD for warm (not hot) data, giving near-DRAM latency at larger capacity, so a page-fault-equivalent miss is far cheaper than a disk fault.
- **NVMe SSD instead of HDD for the swap/storage tier:** For the rare cases that do page out, NVMe reduces the fault penalty by roughly two orders of magnitude versus spinning disk.

- **In-memory caching layer (e.g., Redis/Memcached):** Offload frequently-read query results to a dedicated DRAM-resident cache, reducing reliance on the database's own paging behaviour.

4. Expected Outcome

Combining a right-sized hardware cache hierarchy with a DRAM-first, tiered-memory strategy minimizes reliance on slow disk-backed paging, directly reducing query latency and improving throughput under concurrent load.

Q3. Autonomous Vehicle Data Processing

Problem Statement: *An autonomous vehicle must process sensor data in real time. Evaluate SRAM, DRAM, and MRAM and design a memory hierarchy that maximizes bandwidth and minimizes latency.*

1. Understanding of the Industry Problem

Autonomous vehicles fuse continuous, high-volume streams from LiDAR, radar, and multiple cameras and must run perception/control loops with hard real-time, deterministic latency bounds (safety-critical). The memory system must simultaneously deliver very high bandwidth (for parallel sensor streams and AI inference) and very low, predictable latency (for control-loop decisions), while also safely preserving critical state.

2. Evaluation of Memory Technologies

- **SRAM:** Fastest available (sub-ns to few-ns), volatile, no refresh overhead, but low density and expensive per bit. Ideal for on-chip caches and tightly-coupled scratchpad memory used by real-time control loops (steering/braking) where deterministic, minimal latency is mandatory.
- **DRAM (LPDDR5/HBM):** High density and good bandwidth (especially HBM stacks), suitable as main memory for buffering multi-sensor frame data and running perception/AI inference models. Downsides: periodic refresh overhead and materially higher latency than SRAM, unsuitable for hard-real-time control loops on its own.
- **MRAM:** Non-volatile, near-SRAM speed, very high write endurance, no refresh needed. Ideal for storing safety-critical state (system checkpoints, black-box/event logs) that must survive sudden power loss without the wear-out and slower access of flash.

3. Proposed Memory Hierarchy Design

- **Level 1 – SRAM scratchpad/cache:** Tightly coupled to each perception/control core for deterministic-latency access to the immediate sensor window and control-loop variables.
- **Level 2 – HBM/high-bandwidth DRAM:** Shared main memory pool for buffering incoming multi-sensor (LiDAR/radar/camera) streams and holding neural-network weights/activations for perception models, sized and channel-parallelized to maximize bandwidth.
- **Level 3 – MRAM safety store:** Non-volatile store for rolling checkpoints of vehicle state and critical event logs, written continuously without wear concerns, guaranteeing data survives power interruption for post-incident analysis.
- **Bandwidth/latency balance:** SRAM handles the latency-critical control path; parallel HBM/DRAM channels handle the bandwidth-critical sensor-fusion/AI path; MRAM handles the reliability/persistence requirement — together maximizing bandwidth while minimizing latency where it matters most.

Q4. AI Inference Edge Device

Problem Statement: An edge AI device needs fast model loading with low power consumption. Compare NAND Flash, DRAM, and RRAM and recommend an optimized hierarchical memory structure.

1. Understanding of the Industry Problem

Edge AI devices (IoT sensors, wearables, smart cameras) are battery-constrained and need to load and run inference models quickly after waking from sleep, without the power cost of repeatedly reloading large model weights from slow persistent storage.

2. Comparison of Memory Technologies

- **NAND Flash:** Non-volatile, high density and low cost per bit, so it is commonly used to store full model weights persistently — but it has relatively high read latency and higher energy-per-access for random reads, plus limited write endurance, making frequent reload/update cycles costly in both time and power.
- **DRAM:** Fast, volatile working memory used to hold the active model layers and intermediate activations during inference. Good bandwidth, but it must be continuously refreshed even when idle, so keeping large models resident in DRAM around the clock drains battery.
- **RRAM (ReRAM):** Emerging non-volatile memory with read speeds approaching DRAM and much lower power than NAND Flash. High endurance versus flash, and its analog conductance states can even support in-memory compute (analog multiply-accumulate), reducing data movement for inference.

3. Recommended Optimized Hierarchical Structure

- **RRAM as the primary model store:** Store compressed/quantized model weights in RRAM instead of NAND Flash so the model can be loaded near-instantly on wake with much lower energy than a flash read, eliminating the slow flash-to-DRAM reload step.
- **Small SRAM/DRAM buffer for active computation:** Retain only a minimal DRAM/SRAM working buffer for the layer currently being computed and its activations, rather than holding the entire model in DRAM continuously.
- **RRAM-based compute-in-memory (optional advanced tier):** Where supported, perform matrix-multiply operations directly within the RRAM array to reduce data movement between memory and compute units, further cutting energy per inference.
- **NAND Flash retained for cold storage:** Keep NAND Flash only for firmware, OS, and full model backups/updates that are accessed infrequently, not for runtime model loading.

Q5. High-Performance Gaming Console

Problem Statement: A gaming console suffers frame drops during large scene loading. Analyze L3 cache vs DRAM throughput and propose memory hierarchy enhancements.

1. Understanding of the Industry Problem

Frame drops during large scene loads typically occur when the volume of texture/geometry/asset data that must move from storage through memory to the GPU exceeds what the memory hierarchy can sustain within a frame budget (e.g., 16.6 ms for 60 fps), causing render-pipeline stalls.

2. L3 Cache vs DRAM Throughput Analysis

- **L3 Cache:** Offers very high bandwidth and very low latency, but limited capacity (tens of MB) — nowhere near enough to hold an entire game scene's assets. It is effective only for reusing recently-touched, small, frequently-accessed data (e.g., shader constants, small texture tiles reused across draw calls) across nearby frames.
- **DRAM (e.g., GDDR6/6X):** Provides the large capacity and high aggregate bandwidth needed to hold full scene assets, but at higher latency than L3, and its bandwidth can be saturated when a large volume of new assets is streamed in all at once during a scene load, becoming the bottleneck and causing frame drops.

3. Proposed Memory Hierarchy Enhancements

- **Larger/partitioned L3 (scene cache):** Increase shared L3 capacity and reserve a partition as a 'render cache' for frequently reused textures/geometry, reducing repeated DRAM traffic within and across frames.
- **High-bandwidth unified memory:** Use GDDR6X (or a unified high-bandwidth memory pool shared by CPU/GPU) to raise sustained throughput available for asset streaming.
- **Dedicated high-speed I/O path with hardware decompression:** Stream compressed assets directly from NVMe SSD into memory via a dedicated I/O complex with hardware decompression (similar to modern console architectures), bypassing slower general-purpose I/O paths and reducing load stalls.
- **Predictive/asynchronous asset prefetching:** Prefetch upcoming scene assets during gameplay (before a load screen or camera transition) so streaming overlaps with rendering instead of causing a visible stall.
- **CPU/GPU cache partitioning:** Partition L2/L3 ways between CPU and GPU workloads to prevent contention when both are heavily accessing shared assets during a scene transition.

4. Expected Outcome

Combining a larger, smarter cache tier for reused assets with higher-bandwidth DRAM and predictive, hardware-accelerated streaming reduces the data-movement bottleneck during scene loads, smoothing out frame delivery and eliminating drops.